Guia Prático da Nova Estrutura (O Nosso Padrão de Engenharia)

Para entendermos como o sistema funciona a partir de agora e onde cada coisa deve ficar, utilize este manual didático:

A) `inventario/templates/inventario/` (Apenas HTML Puro)

- **O que deve ficar aqui:** Exclusivamente a estrutura visual da página (marcação HTML, componentes do Bootstrap e as tags de template do Django como `{% for %}`, `{% if %}`).
- **O que NÃO deve ficar aqui:** Nenhuma linha de código JavaScript. Os scripts antigos foram removidos do final das páginas e agora os ficheiros HTML chamam os seus respetivos scripts externos através de tags limpas, como:
- HTML

<script src="{% static 'js/pdv.js' %}"></script>

-
-

B) `static/js/` (Onde ficam os nossos códigos JavaScript)

- *Nota importante de Engenharia:* A pasta `static/admin/js/` é exclusiva do painel interno do próprio Django e **não deve ser mexida**.
- **O que deve ficar na nossa pasta `static/js/`:** Todos os ficheiros de script customizados criados pela nossa equipa para dar interatividade às telas da JB Tintas:
    - `pdv.js`: Contém toda a lógica do carrinho, o cálculo de troco em tempo real, a captura do leitor de código de barras (bipe) e o diálogo com as APIs.
    - `estoque.js`: Controla os modais de cadastro e o cálculo automático do preço de venda com base na margem de lucro.
    - `relatorios.js`: Gere o modal de confirmação para o cancelamento seguro de vendas.

C) `inventario/models/` (Fat Models)

- **O que é:** O modelo de dados da aplicação, agora separado por subpastas (`clientes/`, `produtos/`, `usuarios/`, `vendas/`).
- **Como funciona (Padrão *Fat Model*):** As tabelas não guardam apenas colunas; elas possuem métodos próprios de inteligência. Por exemplo, a classe `Produtos` tem um método chamado `tem_estoque_suficiente()`, e a classe `Usuarios` sabe calcular a sua própria comissão.

- **O "Puxador":** O ficheiro `inventario/models/__init__.py` centraliza e exporta todas estas tabelas para que o Django as reconheça sem erros.

D) `inventario/services/` (Utility Services)

- **O que é:** O "cérebro" financeiro e operacional do sistema, dividido em pastas como `vendas/` e `clientes/`.
- **Como funciona:** É aqui que acontecem as regras de negócio complexas e as transações seguras de banco de dados (protegidas pelo `@transaction.atomic`, garantindo que se algo falhar, nada fica corrompido).
- **Exemplos:** É aqui que reside a função de processar uma nova venda (dando baixa no estoque, calculando comissões e gerando pontos de fidelidade), o cálculo de resgate de pontos e o futuro processador de ficheiros XML da NF-e.

E) `inventario/views/` (Thin Views)

- **O que é:** A camada de apresentação e recepção de pedidos HTTP, dividida por módulos (`vendas/`, `estoque/`, `clientes/`, `configuracoes/` e `autenticacao_views.py`).
- **Como funciona (Padrão *Thin View*):** As vistas tornaram-se "finas" e leves. Elas fazem apenas três coisas:
    1. Recebem o clique ou formulário do utilizador.
    2. Validam quem está a aceder (Controlo de Acesso/RBAC).
    3. Chamam a função correspondente lá no `services.py` e devolvem o resultado para a tela.
- Nenhuma regra matemática pesada ou lógica de banco de dados complexa existe mais aqui.

F) `jb_sistema/` (Configurações Globais)

- **O que fica aqui:** Os ficheiros centrais de configuração do projeto Django.
    - `settings.py`: Contém as definições globais, o caminho atualizado do banco de dados na pasta `banco_de_dados/jb_tintas.db` e o middleware do *WhiteNoise* para gerir ficheiros estáticos na nuvem.
    - `urls.py`: O mapa de rotas da aplicação, atualizado para redirecionar os endereços web diretamente para os novos pacotes modulares de *Thin Views*.